

CAREER



개인 정보

이름	임찬진
이메일	ckswlsdl0916@gmail.com
번호	010-4032-9600
주소	경기도 시흥시 산기대학로 237

학력

2021.03 ~ 2027.02
한국공학대학교 게임공학과 졸업 예정

활동

2026.07 ~ 2026.08
AI 에이전트 기반 언리얼 개발 교육
워크프로세스 자동화 완성과정 이수 및 수료 예정

2025.05
한국게임학회 춘계학술발표대회 논문 게재
논문명 : glTF 2.0 표준 파서, 스키닝 애니메이션, PBR
파이프라인, 오픈소스 물리엔진 서버 최적화 연구

수상 및 자격증

2026.08
졸업작품 우수작 선정
한국공학대학교 게임공학과 / 특성화장학금

2025.03
보드게임 개발자 2급
교육 이수 및 수료

프로젝트

2026.07.01 ~ 2026.08.10
FlickDom  

2025.06.23 ~ 2026.08.09
S.T.L  

2025.11.11 ~ 2025.12.16
TheGreatestNature  

2025.06.01 ~ 2025.06.18
ASSASSIN VS MUTANT  

2022.10.02 ~ 2022.12.06
2DGP Curvy  

스킬

C++20, STL, HLSL

상

GLSL, Unreal, Unity, D3D12

중

OpenGL, AI/MCP, GitHub

중

관련 링크

 깃허브 <https://github.com/AACHANJINAA>

 유튜브 <youtube.com/@나나나나나-c6e>

PORTFOLIO

PORTFOLIO

임찬진

플레이어들에게 행복한 시간을 가질 수 있는
경험을 제공하고 싶은 개발자



포트폴리오 목차

Project	Page	사용 언어	사용 엔진
S.T.L	Page 3 - 9	C++20	D3D12
└ 논문 발표	Page 10	-	-
TheGreatestNature	Page 11	-	UE5
ASSASSIN VS MUTANT	Page 12	-	UE5
2DGP Curvy	Page 13	Python	-
FlickDom	Page 14 - 15	C#	Unity 6
Other Activities			
AI 활용 능력	Page 16 - 18	-	-
마무리	Page 19	-	-



프로젝트
이름

S.T.L (Slay The Lord)

장르

4인 협동 멀티 액션 RPG

설명

파티와 함께 퀘스트를 수행하고 협동 플레이로
보스 몬스터를 공략하는 다크 판타지 게임

개발 인원

4명

담당 역할

클라이언트 그래픽스 파이프라인, glTF 파서,
머티리얼 처리, PBR 셰이더 구현

개발 언어

C++20, STL, HLSL

사용 툴 &
라이브러리

Visual Studio 2022, VS Code, DirectX 12,
ImGui, Json, Git

제작 기간

2025.06.23 ~ 2026.08.09

게임 콘텐츠 & 시스템

핵심 게임 플레이

- 최대 4명의 플레이어가 파티를 맺고 거대한 보스 몬스터와 전투
- 어그로 수치 기반 실시간 타깃 전환과 역할 분담
- 대시, 가드, 타격 판정을 프레임 단위로 동기화



그래픽스 파이프라인

렌더링 구현

- glTF 2.0 표준 파서 기반 메시와 머티리얼 데이터 처리
- PBR 파이프라인과 셰이더 렌더링 구현
- HLSL 셰이더 기반 조명 및 표면 표현 구성



glTF PBR 코드 구현

그래픽스 파이프라인

- glTF 2.0 Material 데이터를 엔진용 MaterialInfo로 직접 변환
- BaseColor, Normal, ORM, Emissive, Occlusion 텍스처와 UV Transform 지원
- 텍스처 존재 여부를 플래그로 전달해 하나의 HLSL PBR 셰이더에서 처리
- 플레이어 식별용 Phantom Aura를 Fresnel 기반 색상 연출로 추가

glTF
Parse

D3D12
Bind

HLSL
Render

01

Material Parser

glTF 머티리얼을 엔진 데이터로 변환

BaseColor, Normal, ORM, Emissive, Occlusion 텍스처와 UV 채널 및 Transform 값을 함께 파싱했습니다.

```
load_materials_from_gltf()
```

02

D3D12 Binding

텍스처 플래그와 상수를 GPU로 전달

텍스처가 없는 에셋도 깨지지 않도록 존재 여부 플래그와 기본값을 셰이더 상수 버퍼에 묶었습니다.

```
bind_material()
```

03

HLSL Shader

PBR 렌더링과 게임 전용 오라 연출

직접광, IBL, Shadow, Emissive를 합산하고 플레이어 ID에 따라 Fresnel 기반 Phantom Aura 색상을 적용했습니다.

```
PS_GLTF() + ApplyPhantomAura()
```

중요 코드 설명 1

리소스 전달 구조

C++20과 STL 컨테이너를 활용해 glTF의 메시, 텍스처, 머티리얼 데이터를 엔진 내부 자료구조로 정리하고 D3D12 렌더링 단계로 전달했습니다.

unordered_map 캐싱

문자열 키 기반 캐시로
리소스 재사용 구조화

vector 파싱 배열

glTF 인덱스 순서를 유지해
참조 관계를 보존

filesystem 경로 조합

상대 URI를 모델 기준
절대 경로로 해석

01. Resource Cache with STL

ResourceManager.h

```
std::unordered_map<std::string, std::shared_ptr<Mesh>> _meshes;  
std::unordered_map<std::string, TextureInfo> _textures;  
std::unordered_map<std::string, MaterialInfo> _materials;  
  
std::vector<TextureInfo*> _pending_textures;  
std::deque<std::shared_ptr<Mesh>> _pending_meshes;
```

02. glTF Material Parse

ResourceManager.cpp

```
std::vector<std::string> images_uris;  
std::vector<std::string> texture_source_uris;  
std::filesystem::path base_path = std::filesystem::path(file_path).parent_path();  
  
std::filesystem::path full_texture_path = base_path / texture_source_uris[tex_idx];  
path_member = full_texture_path.string();
```


중요 코드 설명 2

Shader 활용 포인트

ResourceManager에서 전달한 텍스처 존재 여부, 머티리얼 상수, UV Transform 값을 셰이더에서 조건 처리해 PBR 색상과 게임 전용 효과를 최종 픽셀 색상으로 합산했습니다.

Material CBuffer	cbMaterial register(b2)로 머티리얼 상수 전달
Texture Flag	텍스처 존재 여부로 샘플링 분기 처리
Final Color	Lighting/IBL 결과에 오라 후처리 합산

01. cbMaterial Structure

Gltf_Shader.hlsl

```
cbuffer cbMaterial : register(b2)
{
    float4 BaseColorFactor;
    float4 EmissiveAndMetallicFactor;
    float RoughnessFactor;
    int HasMetallicRoughnessTexture;
    ....

    float2 BaseColorUVOffset, BaseColorUVScale;
    float2 MetallicRoughnessUVOffset;
    float2 MetallicRoughnessUVScale;
    ....
};
```

02. Material Value Usage

Gltf_Shader.hlsl

```
roughness = lerp(RoughnessFactor, orm.g * RoughnessFactor, hasMetTex);
metallic = lerp(MetallicFactor, orm.b * MetallicFactor, hasMetTex);
float3 emissive = finalEmissive.rgb;

float4 litColor = Lighting(..., albedo, metallic, roughness, ao);
float3 iblColor = CalculateIBL(N, V, albedo, metallic, roughness, ao);
finalColor = (litColor.rgb * shadowFactor) + iblColor + finalEmissive;

if (g_otherplayerid > -1)
    finalColor = ApplyPhantomAura(finalColor, N, V);
```

개발 결과 및 성과

결과와 성과

결과

- D3D12 기반 자체 렌더링 파이프라인 구성
- glTF 에셋을 실제 게임 화면에 안정적으로 출력
- BaseColor, Normal, ORM, Emissive 텍스처를 통합한 PBR 셰이더 구조 구현

성과

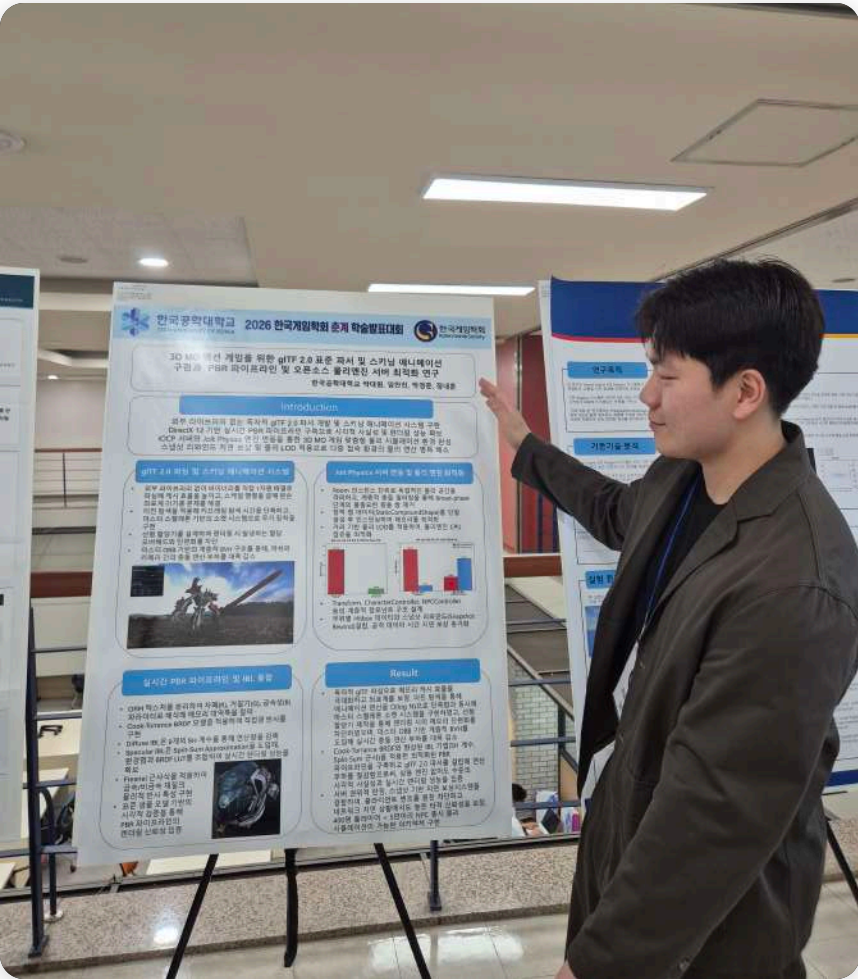
- 그래픽스 구현 내용을 논문 주제와 연결해 기술 구현 과정 문서화
- 졸업작품 우수작 선정 및 특성화장학금 수상
- 한국게임학회 추계학술대회 포스터 논문 전시



논문 발표로 이어지는 그래픽스 구현

S.T.L에서 직접 구현한 glTF 파서, PBR 셰이더, D3D12 렌더링 파이프라인을 기술 주제로 정리했고, 다음 페이지에서 논문 발표 내용으로 연결됩니다.

논문 발표



한국게임학회 춘계 학술발표대회

논문명 : 3D MO 액션 게임을 위한 glTF 2.0 표준 파서 및 스키닝 애니메이션 구현과 PBR 파이프라인 및 오픈소스 물리엔진 서버 최적화 연구

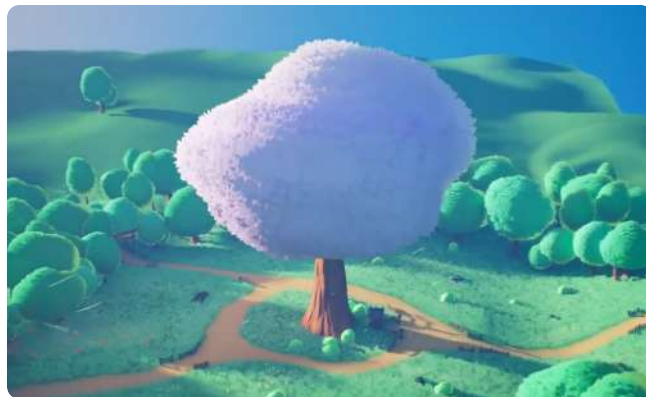
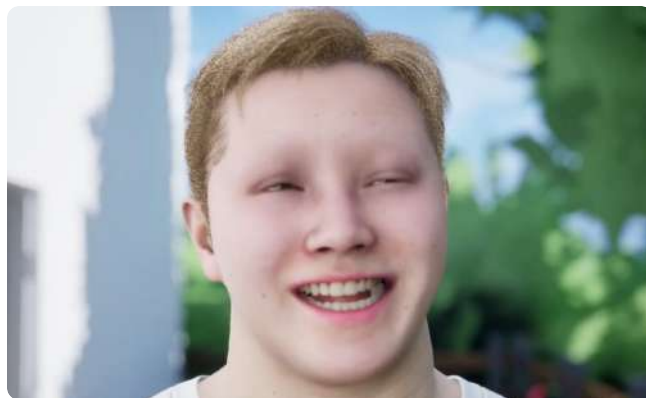
담당 파트 : 그래픽스 구현
glTF 2.0 표준 파서와 PBR 파이프라인 구현 내용을 중심으로 정리했습니다.

핵심 구현
외부 라이브러리 의존을 줄이고 DirectX 12 환경에서 메시와 머티리얼 데이터를 직접 처리했습니다.

PBR 파이프라인
ORM 텍스처, Cook-Torrance BRDF, IBL, Fresnel 기반 표현을 활용해 실시간 렌더링 품질을 개선했습니다.

TheGreatestNature

Team Project



프로젝트
이름

TheGreatestNature

장르

단편 시네마틱 영상

설명

자연을 바라보며 치유되는 감정을
MetaHuman과 Sequencer로 연출한 UE5
시네마틱

개발 인원

팀 프로젝트

담당 역할

MetaHuman 리타게팅, 표정 제어, Camera
Rail, Sequencer 컷 편집, 조명 구성

개발 언어

Blueprint

사용 툴 &
라이브러리

Unreal Engine 5, MetaHuman,
Sequencer, Git

제작 기간

2025.11.11 ~ 2025.12.16

ASSASSIN VS MUTANT



프로젝트
이름

ASSASSIN VS MUTANT

장르

3인칭 3D 액션 게임

설명

암살자 캐릭터가 돌연변이 적을 상대하는
UE5 기반 액션 전투 프로젝트

개발 인원

1명

담당 역할

전투 액션, 무기 스위칭, 적 AI, UI, 승패 흐름,
연출 이미지 적용

개발 언어

Blueprint

사용 툴 &
라이브러리

Unreal Engine 5, Git

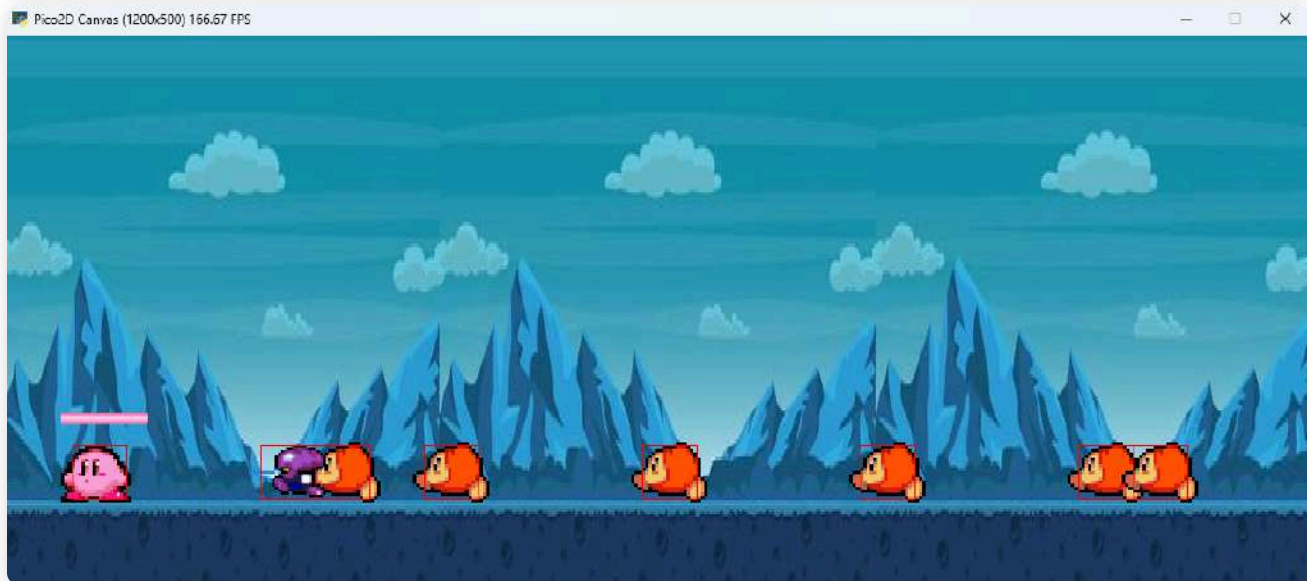
제작 기간

2025.06.01 ~ 2025.06.18

2DGP Curvy



Personal Project



프로젝트
이름

2DGP Curvy

장르

2D 횡스크롤 액션 게임

설명

Python 기반으로 게임 루프, 충돌 처리, 캐릭터 상태 전이를 직접 구현한 개인 프로젝트

개발 인원

1명

담당 역할

플레이어 조작, FSM, 공격 판정, 몬스터 이동, 씬 전환 구조 구현

개발 언어

Python

사용 툴 &
라이브러리

pico2D, VS Code, Git

제작 기간

2022.10.02 ~ 2022.12.06

FlickDom (AI VIBE CODING)



Team Project



프로젝트
이름

장르

설명

FlickDom (AI VIBE CODING)

1 VS 1 전략 플릭 보드게임

디스크를 튕겨 보드 칸을 차지하고 카드
패턴을 완성해 점수를 겨루는 물리 기반 파티
게임

개발 인원

팀 프로젝트

담당 역할

Unity MCP 기반 구현 반복, 멀티플레이 흐름,
사운드 제작 및 적용, 기획 총괄

개발 언어

C#

사용 IDE

Unity 6 Editor, VS Code, Blender,
Substance 3D Designer

AI 모델

GPT-5 Codex, Unity MCP, Blender MCP,
Substance MCP, VARCO AI Sound

제작 기간

2026.07.01 ~ 2026.08.10

게임 시스템



목표

3개의 디스크로 보드 칸을 점령하고 카드 패턴을 완성합니다.



조작

디스크를 선택한 뒤 드래그 방향과 힘으로 발사해 다음 턴 전략을 만듭니다.



멀티플레이

Host가 방을 생성하고 Join Code를 공유하면 Client가 접속합니다.



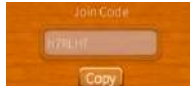
시작 전 준비
Host / Client 역할 확인



Join Code 입력
호스트 코드로 같은 방 접속



상태 표시
접속 후 역할 상태 확인



초대 코드 전달
최신 코드 공유 후 시작

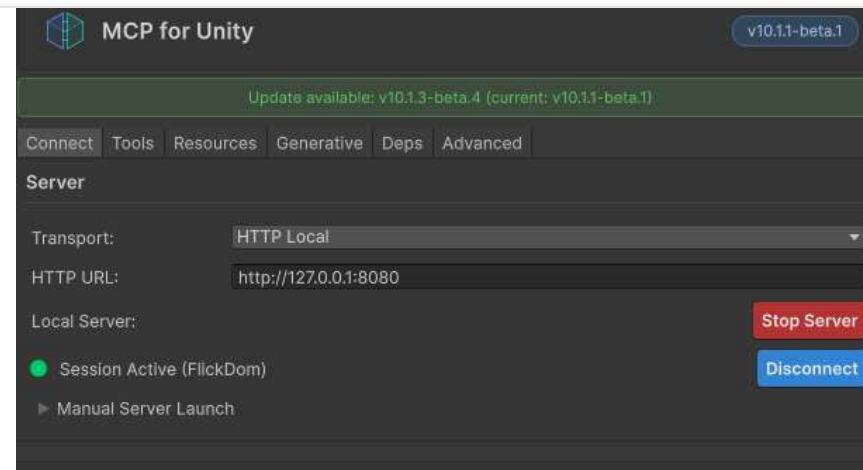
AI/MCP 활용으로 이어지는 제작
과정

Unity MCP, Blender MCP, Substance MCP와 AI 사운드 제작 흐름을 함께 활용해 리소스 정리와 구현 반복 속도를
높였습니다.

AI/MCP 기반 개발 1

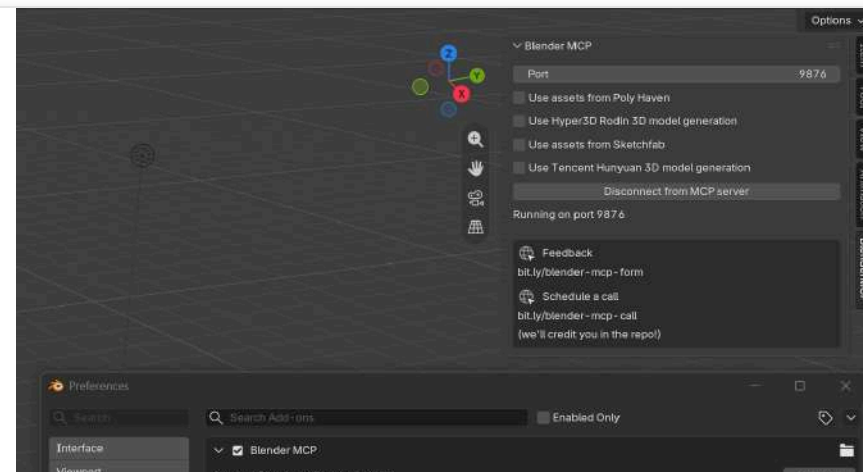
Unity MCP

Unity Editor와 AI 에이전트를 연결해 씬, 오브젝트, 컴포넌트 작업을 빠르게 확인하고 반복했습니다.



Blender MCP

보드, 디스크, 소품 리소스의 비율과 배치 목적을 정리하며 3D 제작 과정을 보조했습니다.



AI/MCP 기반 개발 2

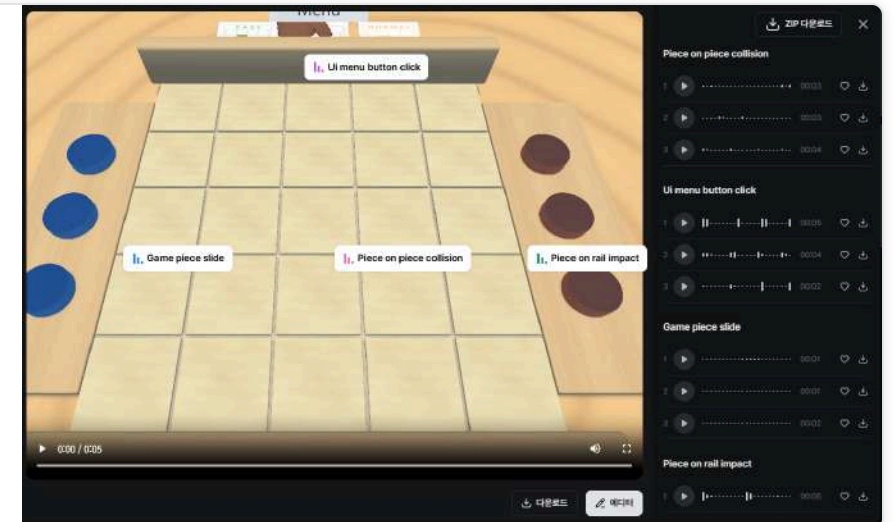
Substance MCP

보드, 디스크, 셀 타일의 머티리얼 방향과 UV 스케일을 정리하고 Unity 적용 흐름을 맞췄습니다.



VARCO AI Sound

버튼 클릭, 점수 획득, 실패, 승리 등 상황별 효과음을 제작해 플레이어 피드백을 강화했습니다.



MD 기반 프로젝트 뼈대

MD 기반 프로젝트 뼈대 및 모듈화

AI(MCP)가 작업 맥락을 빠르게 파악해 효율성을 극대화하도록 MD 파일을 적극 활용했습니다. 무분별한 파일 생성은 AI 학습을 저해하므로 Architecture.md를 기준으로 목적에 맞는 세부 문서를 조립했습니다.

작업 규칙 및 아키텍처

Architecture.md (기준 구조도)

AGENTS.md, CONTRIBUTING.md (작업 규칙)

DEVELOP_LOG.md (구현 이력 관리)

게임 기획 및 기능 모듈화

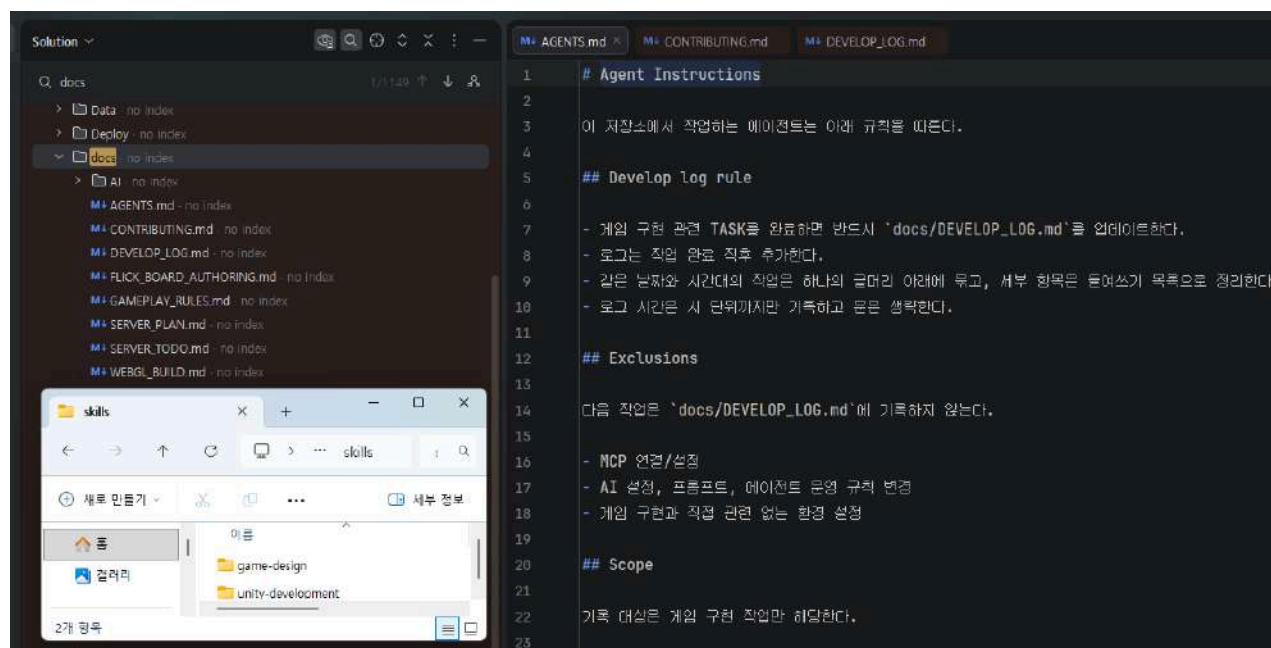
GAMEPLAY_RULES.md (핵심 게임 규칙)

SERVER_PLAN.md (멀티플레이 동기화 계획)

리소스 제작 및 빌드 가이드

FLICK_BOARD_AUTHORIZING.md (에셋 제작 기준)

WEBGL_BUILD.md (배포 및 빌드 설정)



CONTACT

감사합니다.

이름 임찬진

전화번호 010-4032-9600

이메일 ckswlsdl0916@gmail.com

